OFICINA MECÂNICA INTELIGENTE

Guia de Estudo da Arquitetura & Código

Manual Completo de Navegação, Camadas e Fundamentos Técnicos para Apresentação

PHP 7.2 | Laravel 7 | Clean Architecture | SOLID & Design Patterns | MySQL / MariaDB | Bootstrap 4 | Controle de Acesso RBAC

Este guia foi elaborado para que você possa **estudar, compreender e explicar** com segurança toda a estrutura do sistema **Oficina Laravel**. Cada módulo foi desenhado com responsabilidades bem definidas, seguindo as melhores práticas da engenharia de software corporativa.

🏛️ 1. Filosofia Arquitetural do Sistema

O sistema adota o padrão de **Arquitetura em Camadas (Layered Architecture)** associado aos princípios **SOLID**, garantindo que regras de negócio, persistência de banco e interfaces de tela nunca fiquem acopladas.

O Ciclo de Vida de uma Ação (Fluxo de Dados de Ponta a Ponta)

```
[ Navegador / Usuário ] │ (Requisição HTTP: GET, POST, PUT, DELETE) ▼ [ routes/web.php ] ──────► Identifica o padrão da rota e a URL acessada │ ▼ [ Middleware: VerificarPerfil ] ──► Valida permissões RBAC (Admin, Mecânico, Atendente) │ ▼ [ Form Request: Validação ] ─────► Limpa máscaras (prepareForValidation) e valida regras │ ▼ [ Controller ] ──────────────────► Skinny Controller: Apenas orquestra entrada e saída │ ▼ [ Service (Regra de Negócio) ] ─► Executa cálculos, regras de estoque e transações │ ▼ [ Repositório (Interface/Eloquent) ] ─► Isola as queries e acesso aos dados │ ▼ [ Model (Eloquent ORM) ] ──────► Mapeia entidades e relacionamentos no MySQL │ ▼ [ Blade View (Interface HTML) ] ─► Renderiza a tela responsiva com Bootstrap 4
```

🗺️ 2. Mapa Rápido: Onde Encontrar as Informações no Código

Consulte a tabela abaixo sempre que precisar localizar um arquivo rapidamente durante o estudo:

O que você quer entender ou alterar?	Caminho do Arquivo no Projeto	Responsabilidade Principal
Rotas e Endpoints HTTP	`routes/web.php`	Mapeia URLs para Controllers e aplica middlewares.
Controle de Perfis (RBAC)	`app/Http/Middleware/VerificarPerfil.php`	Barra acessos não autorizados por perfil de usuário.
Validação e Sanitização de Dados	`app/Http/Requests/*.php`	Remove pontuações de CPF/CNPJ e valida campos.
Controladores Web	`app/Http/Controllers/*.php`	Recebem a requisição e retornam views ou redirects.
Regras de Negócio e Cálculos	`app/Services/*.php`	Cálculo de orçamentos, estoque e fluxo de O.S.
Abstração do Banco (Repositórios)	`app/Repositories/Eloquent/*.php`	Consultas SQL isoladas via Eloquent ORM.
Injeção de Dependências (IoC)	`app/Providers/RepositorioServiceProvider.php`	Vincula Interfaces aos Repositórios e Serviços.
Modelos e Relacionamentos	`app/Models/*.php`	Classes Eloquent (hasMany, belongsTo).
Tabelas do Banco de Dados	`database/migrations/*.php`	Estrutura das 10 tabelas relacionais MySQL.
Layout Base & Barra Lateral	`resources/views/layouts/app.blade.php`	Estrutura visual, sidebar retrátil e alertas.
Tabela Dinâmica de Orçamentos	`resources/views/orcamentos/criar.blade.php`	JavaScript para adicionar peças e serviços.
Prontuário Histórico do Carro	`resources/views/veiculos/historico.blade.php`	Linha do tempo cronológica das manutenções.

🔍 3. Mergulho Detalhado em Cada Camada

3.1. Roteamento & Matriz de Permissões (RBAC)

No arquivo `routes/web.php`, as rotas são protegidas e segmentadas por prefixos lógicos. A segurança do sistema opera em 3 níveis:

- **ADMINISTRADOR**: Acesso total irrestrito (gerencia funcionários, visualiza faturamento, cadastra dados e realiza exclusões).
- **ATENDENTE**: Foco comercial (cadastra clientes, registra veículos, elabora orçamentos e colhe aprovação do cliente). Bloqueado de alterar laudo técnico ou gerenciar funcionários.
- **MECÂNICO**: Foco operacional (visualiza fila de O.S., atualiza andamento para diagnóstico/execução, requisita peças e finaliza a manutenção com laudo técnico).

> **Implementação no Código:** Veja como o middleware `app/Http/Middleware/VerificarPerfil.php` intercepta as requisições antes que qualquer controller seja acionado, retornando HTTP 403 amigável caso o usuário não tenha o nível necessário.

3.2. Validação & Sanitização com Form Requests

Para não sobrecarregar os Controllers com `$request->validate()`, foram criadas classes dedicadas na pasta `app/Http/Requests/`. Cada classe possui:

- `authorize()`: Retorna `true` confirmando a autorização da ação.
- `prepareForValidation()`: Realiza a limpeza de máscaras de entrada (ex: remove pontos e traços de CPF/CNPJ antes de validar unicidade).
- `rules()`: Define tipos de dados, tamanhos máximos, campos obrigatórios e chaves únicas no banco.
- `messages()`: Retorna mensagens de erro em português claro para exibição na tela.

```
// Exemplo de Sanitização em app/Http/Requests/ClienteRequisicao.php
protected function prepareForValidation()
{
    if ($this->has('cpf_cnpj')) {
        $this->merge([
            'cpf_cnpj' => preg_replace('/[^0-9]/', '', $this->cpf_cnpj)
        ]);
    }
}
```

3.3. Controladores Magros (Skinny Controllers)

Localizados em `app/Http/Controllers/`. Um erro comum em projetos PHP é colocar regras de negócio dentro do Controller. Neste projeto, os Controllers funcionam estritamente como **coordenadores de tráfego**:

```
// Exemplo em app/Http/Controllers/ClienteController.php
public function armazenar(ClienteRequisicao $request)
{
    $this->clienteServico->cadastrarCliente($request->validated());
    return redirect()->route('clientes.index')->with('sucesso', 'Cliente cadastrado com sucesso!');
}
```

3.4. Camada de Negócio (Services) — A Inteligência do Sistema

Localizada em `app/Services/`. Aqui está o coração funcional da oficina:

1. Orçamento Inteligente (`OrcamentoServico.php`)

- Calcula automaticamente os subtotais de peças físicas e serviços de mão de obra.
- Aplica abatimento de descontos promocionais e gera o valor final total.
- **Conversão Atômica em O.S.:** Quando o cliente aprova o orçamento, o método `converterEmOrdemServico()` abre uma transação de banco de dados (`DB::transaction`), cria a O.S., clona os itens, debita as peças do almoxarifado e registra a entrada no histórico do carro.

2. Almoxarifado & Ordens de Serviço (`OrdemServicoServico.php`)

- **Bloqueio de Saldo:** Valida se há peças físicas disponíveis antes de aprovar a inclusão na manutenção.
- **Garantia de Estorno:** Se uma O.S. for cancelada, todas as peças reservadas voltam automaticamente para o estoque físico.
- **Emissão de Laudo:** Ao finalizar a O.S., registra o parecer do mecânico e consolida o evento na linha do tempo do veículo.

3.5. Camada de Repositórios & Inversão de Dependência (SOLID)

O sistema utiliza o **Padrão Repository** com interfaces:

- **Contratos (Interfaces):** Em `app/Repositories/Contratos/` (ex: `OrcamentoRepositorioInterface.php`).
- **Implementações Eloquent:** Em `app/Repositories/Eloquent/` (ex: `OrcamentoRepositorio.php`).
- **IoC Service Provider:** Em `app/Providers/RepositorioServiceProvider.php`, o container do Laravel faz o bind:

```
// app/Providers/RepositorioServiceProvider.php
$this->app->bind(OrcamentoRepositorioInterface::class, OrcamentoRepositorio::class);
$this->app->bind(OrcamentoServicoInterface::class, OrcamentoServico::class);
```

> **Vantagem Técnica:** Os serviços nunca conversam diretamente com o banco de dados. Eles chamam a interface. Se no futuro o banco mudar ou for necessário criar testes unitários com mocks, a lógica permanece 100% intacta.

3.6. Banco de Dados Relacional (10 Tabelas Normalizadas)

Localizado em `database/migrations/`. Todas as tabelas foram modeladas na **3ª Forma Normal (3FN)**:

1. `usuarios`: Contas com senha criptografada em bcrypt e perfil de acesso.
2. `clientes`: Dados cadastrais com CPF/CNPJ único.
3. `veiculos`: Frota automotiva vinculada ao cliente, com placa e quilometragem.
4. `pecas`: Estoque com código de barras, preço de custo, venda e estoque mínimo.
5. `servicos`: Catálogo com procedimentos de mão de obra e tempo estimado.
6. `orcamentos`: Propostas com validade, descontos e totalização calculada.
7. `itens_orcamento`: Itens polimórficos de peça ou serviço na proposta.
8. `ordens_servico`: Acompanhamento da oficina (Aberta, Em Execução, Finalizada, etc.).
9. `itens_ordem_servico`: Peças e procedimentos reais executados no veículo.
10. `historicos_veiculo`: Prontuário médico automotivo com histórico de odômetro e laudos.

3.7. Interface com o Usuário (Blade Views & Front-End)

- **Layout Mestre (`layouts/app.blade.php`):** Possui sidebar profissional retrátil, ícones FontAwesome, menus condicionais por perfil e mensagens flash de feedback.
- **Componente Dinâmico (`orcamentos/criar.blade.php`):** Utiliza JavaScript Vanilla para adicionar linhas de peças ou serviços dinamicamente, alternando os campos e preenchendo automaticamente preços e descrições vindos do catálogo.
- **Prontuário Histórico (`veiculos/historico.blade.php`):** Apresenta uma linha do tempo vertical elegante listando todas as passagens do veículo, odômetro aferido e serviços executados.

🗓️ 4. Roteiro Recomendado de Estudos (Passo a Passo)

Siga esta ordem didática para entender o sistema sem se perder:

1. **Passo 1 — Banco & Entidades:** Abra `database/migrations/` e depois `app/Models/` para ver como os dados se relacionam.
2. **Passo 2 — Contratos & Repositórios:** Abra `app/Repositories/Contratos/` e `app/Repositories/Eloquent/` para ver como as buscas são abstraídas.
3. **Passo 3 — Inteligência de Negócio:** Leia `app/Services/OrcamentoServico.php` e `OrdemServicoServico.php` para dominar os cálculos e o fluxo de O.S.
4. **Passo 4 — Roteamento & Segurança:** Abra `routes/web.php` e `app/Http/Middleware/VerificarPerfil.php` para ver o fluxo de requisição e RBAC.
5. **Passo 5 — Interface do Usuário:** Abra `resources/views/orcamentos/criar.blade.php` e `veiculos/historico.blade.php` para analisar o front-end.

🎯 5. Como Apresentar o Projeto com Autoridade Técnica

> ***1. Sobre a Arquitetura:***
> *"O projeto adota Clean Architecture e SOLID. Em vez de acoplar regras de cálculo no Controller, os Controllers apenas recebem a requisição, os Form Requests validam e limpam dados, os Services executam a regra de negócio e os Repositories gerenciam a persistência."*

> ***2. Sobre a Integridade Operacional:***
> *"A conversão de um orçamento em Ordem de Serviço ocorre dentro de uma transação atômica (DB::transaction). Se o estoque de uma peça for insuficiente, a transação é revertida integralmente, evitando inconsistências financeiras e operacionais na oficina."*

> ***3. Sobre a Segurança e RBAC:***
> *"O sistema implementa Controle de Acesso Baseado em Perfis (RBAC) com níveis para Administrador, Mecânico e Atendente, garantindo que mecânicos não alterem orçamentos comerciais e atendentes não encerrem ordens sem laudo técnico emitido."*